Track 4: Quantum Error Correction

From the Steane code to LDPC decoders

Qiskit Fall Fest Hackathon · Oct 9, 2026 · IISER Bhopal (EECS Club and Physics Club)

Read this first. You have until Oct 17, a laptop, and AI assistants. The ten problem statements are suggestions, not a menu you must order from. You may pick one, extend one, combine several, or invent something more ambitious. Original, advanced and cross-track projects are welcome and will be judged on the same criteria.

What we reward. A project that works, that you can explain line by line, and that is honestly compared against a classical alternative. Using AI to write code is allowed and encouraged; being unable to explain or verify that code is not.

How to read the problem statements

Every problem has the same parts: a short background, the problem itself, the core tasks (what a complete submission does), stretch goals (what makes it excellent), and a hint on tools. Treat the core tasks as the minimum, not the ceiling.

Simulators first, hardware as a bonus. Everything here can be done on a simulator (Qiskit Aer or the statevector tools). Running a small version on real IBM hardware, with a clear comparison against the simulator, earns extra credit in any track.

This track contains: **Problem 7, Problem 8**.

Problem 7: A logical qubit with the Steane code

Background

The Steane code protects one logical qubit using seven physical qubits, and corrects any single error of any type (bit flip, phase flip or both). It is built from the classical Hamming code, which makes it a perfect first example of how classical ideas become quantum ones. Its structure also allows several logical gates to be applied transversally, meaning one gate per physical qubit, so a single faulty gate cannot spread through the block.

Problem

Build, test and use a working Steane-code logical qubit in Qiskit, and show that it really does protect information.

Core tasks (the minimum for a complete submission)

1. Build the encoding circuit that turns one qubit into the seven-qubit logical state, and draw it in the Qiskit Composer or with Qiskit's circuit drawer. Explain the stabilisers it uses.
2. Build the syndrome extraction circuit with ancilla qubits, and the classical lookup that maps each syndrome to a correction.
3. Inject each of the 21 possible single-qubit errors (X, Y or Z on each of the 7 qubits) and show that every one is detected and corrected. Present this as a table.
4. Demonstrate at least one transversal logical gate (for example logical X, Z, H or CNOT between two blocks) and check it on the logical states.

Stretch goals (what makes it excellent)

- Show what happens with two errors: the code should fail, so measure the logical error rate against the physical error rate with a noise model and find the break-even point.
- Use mid-circuit measurement and feed-forward so the correction runs inside the circuit.
- Add flag qubits or repeated syndrome measurements, so errors in the syndrome circuit itself are not fatal.
- Use your logical qubit in a larger task (see the combined-project ideas), such as a protected QKD or Grover step.

Hints on tools

Qiskit's if_test handles classically controlled corrections, and the Aer simulator supports noise models. Keep a clear table of the stabiliser generators; most mistakes come from mixing up which qubits each one touches.

Problem 8: LDPC decoding from scratch

Background

Low-density parity-check (LDPC) codes use a sparse parity-check matrix, and are decoded by passing messages along a graph (the Tanner graph) until the nodes agree. They power modern communications such as 5G and Wi-Fi, they are used to correct errors in QKD key generation, and their quantum cousins are a leading route to practical, low-overhead error correction.

Problem

Implement an LDPC decoder, measure its performance against noise, and understand why it works and where it fails.

Core tasks (the minimum for a complete submission)

1. Construct a sparse parity-check matrix (for example a regular LDPC code with a few hundred bits), and draw a small example as a Tanner graph.
2. Implement belief propagation decoding (the sum-product or min-sum variant) for bits sent through a binary symmetric channel.
3. Plot the frame error rate and bit error rate against the channel error probability, and identify the threshold region where decoding starts to fail.
4. Explain what the decoder is doing in one clear figure, such as how the messages evolve over iterations on a failing and a succeeding example.

Stretch goals (what makes it excellent)

- Compare belief propagation with a simpler decoder (bit flipping) and with an improved one, and report the difference.
- Move to quantum codes: build a hypergraph-product or bivariate bicycle style code, and decode with belief propagation combined with ordered statistics decoding (BP+OSD). Why does plain belief propagation struggle on quantum codes?
- Implement the decoder in a hardware-friendly way, for example fixed-point arithmetic, and report the cost of lower precision.
- Use your decoder inside the reconciliation step of a QKD protocol (see the combined-project ideas).

Hints on tools

The open-source ldpc Python package can serve as a reference to check your own implementation, and stim simulates quantum error correction circuits efficiently. A decoder you wrote yourself and verified against a reference is worth more than one imported in a single line.

Go beyond: combine tracks or invent your own

The most impressive entries will not stop at one box. None of these is required, and you are free to propose something entirely different.

Ideas that involve Track 4

Idea	Tracks combined	What makes it interesting
A logical qubit inside a QKD protocol	Steane code and BB84 or BBM92	Does encoding the transmitted qubits lower the QBER, and at what cost in qubits?
LDPC reconciliation in QKD post-processing	LDPC decoding and QKD	Replace a simple error-correction step with your own decoder, and plot final key rate against QBER.
A neural-network decoder	QNN and error correction	Train a quantum or classical network to decode Steane syndromes, and compare with the lookup table.
Logical qubits inside an optimiser	Steane code and QAOA or VQE	Does error protection improve the answer on a noisy simulator, or does the overhead cancel the gain?
QAOA for code design	QAOA and error correction	Search for good parity-check matrices or qubit layouts as an optimisation problem.
Decoder as a hardware design	LDPC decoding and FPGA or fixed-point design	Quantify the accuracy lost by low-precision arithmetic, as a first step toward hardware.

Open invitation. If you have your own idea, such as a quantum game, an algorithm tutor, a benchmark of a quantum algorithm from our workshop (Grover, phase estimation, the QFT), or a tool that makes any of the above easier for the next student, propose it. Write three sentences on the problem, the method, and how you will know it worked, and check with the organisers early.

How to scope a combined project. Get the smallest version of each part working separately first, then connect them. A small, honest, working combination beats an ambitious one that does not run.

What every team submits

Whichever problem you choose (or invent), your submission has the same four parts:

- A code repository or notebook that runs from a clean start, with versions of the packages listed.
- A write-up PDF (two to four pages) covering the problem, method, results and an honest limits section.
- At least one figure that shows the main result.
- A short AI-use note: which tools you used, one thing the AI got wrong, and how you caught it.

Bonus. Bonus points are available for a hardware run on a real IBM backend, with a clear comparison against the simulator.

Presentation. Submissions close on Oct 17, 2026, 12:00 noon. After reviewing them, the organisers invite selected teams to present on Oct 18, 2026: a five-minute demo followed by questions, where every team member must be able to take questions. Teams that are not invited are still fully judged on their submission.

Pitch deck (advised). Teams are selected from their submissions alone; a pitch deck is not part of the submission and is not used for selection. However, selected teams will have less than 24 hours between the invitation and the presentation, so we strongly advise preparing a short pitch deck for your idea in advance.

Full submission rules are in *docs/guidelines_and_submission_rules.pdf*.

How you are judged

The same scoring applies to every problem and to original projects alike.

Criterion	Points	What judges look for
Correctness and verification	30	Results checked against an exact or classical reference, tests, sensible error bars
Understanding and insight	25	Clear explanation of why the method works or fails, shown in the write-up (and in the live questions for invited teams)
Honest comparison and limits	20	A fair classical baseline, stated assumptions, no unsupported claims of quantum advantage
Ambition and originality	15	Stretch goals, combined tracks, or a new idea beyond the suggested statements
Clarity of code and write-up	10	Readable, reproducible code and a clear write-up and story
Total	**100**	Bonus points are available on top for a real IBM hardware run (see Bonus).

> **On using AI.** AI assistants are welcome at every step. The AI-use note and the presentation questions exist because the person submitting is responsible for the result: you should be able to explain each circuit, formula and plot, and show how you verified it.